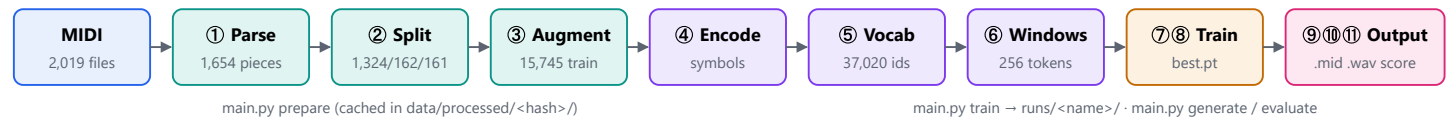


The idea: treat music like text. Every note, chord or silence becomes a *token*, and an LSTM learns to predict the next token. To compose, it predicts one token, appends it, and repeats. All numbers below are the final model (v5, configs/adl_mixed.yaml).



1 Parse and quantize

src/data/parse.py

MIDI file -> Piece (list of NoteEvent(pitch, start, duration, velocity), times in beats)

- Reads with `pretty_midi`; merges all non-drum tracks into one stream (all ADL tracks are piano).
- Converts seconds -> beats through every tempo change (`tempo_map`), then snaps start and length to a 16th-note grid (0.25 beat).
- Drops: pieces with a time signature other than 4/4, 3/4, 2/4; notes < 0.125 beat; pieces < 50 notes. Merges the same pitch struck twice at once.
- Style label = top folder name (Classical / Jazz / Blues). 11,073 ADL files -> 2,019 in these genres -> **1,654 kept** (1.88 M notes).

2 Split by piece

augment.split_pieces

- Fixed shuffle (seed 1337) -> 80 / 10 / 10 % train / val / test, by *whole piece*.
- Drops val/test pieces that duplicate a training piece, even transposed -> **1,324 / 162 / 161**.
- Done **before** augmenting, so no version of a held-out song leaks into training.

3 Augment (train only)

augment.transpose_augment

- Adds copies shifted -6 ... +6 semitones (13 keys). A copy is dropped if any note leaves piano range 21-108.
- 1,324 -> **15,745** training pieces. Teaches patterns independent of key.

4 Encode to symbols

src/data/encode.py

Piece -> ["<STYLE:Jazz>", "60.64.67", "<REST:2>", "62", ...]

- **note_chord**: all notes starting together = one symbol, pitches joined by dots. Each note/chord symbol advances the clock one 16th.
- Silence = one run-length rest <REST:k> (k = 2...16 steps; <REST> = 1). Cut rests from 56 % to 36 % of tokens.
- No duration in the symbol (it would make 363,455 symbols). Durations are rebuilt at playback (step 10).
- Other schemes (config switch): `interval`, `pitch_duration`, `event`.

5 Vocabulary

src/data/vocab.py

- Symbol -> integer id, from the **training split only**, in `sorted()` order. Reserved: <PAD> 0 <UNK> 1 <BOS> 2 <EOS> 3 <REST> 4.
- Symbols seen < 10 times -> <UNK> (~4 % of tokens). Result: **37,020** symbols, 12.8 M training tokens.
- Saved as `vocab.json` beside the model; the model never runs with a different vocabulary.

6 Windows and batches

src/data/dataset.py

- Each piece is cut into 256-token windows every 128 tokens (half overlap), plus one window aligned to the end. Never crosses into another piece.
- **Teacher forcing**: $y = x$ shifted one step left, so every position is a target (256 per window).
- Shuffled batches of 48 -> tensors `x`, `y` : [48, 256].

7 Model: MusicLSTM

src/models/lstm.py

Layer	Shape	Params
Embedding	37,020 -> 256	9.5 M
LSTM x 3 (dropout 0.3 between)	256 -> 512 -> 512 -> 512	5.8 M
Dropout 0.3 · Linear · ReLU	512 -> 256	0.13 M
Adaptive softmax (head 2,000 · 8,000 · 27,020)	256 -> 37,020	1.48 M
Total		16.87 M

- Output [B, 256, 37,020]: a probability for every symbol at every position.
- Init: orthogonal recurrent weights, forget-gate bias 1 (the cell starts out remembering).

8 Training loop

src/train.py

- Per batch: forward -> cross-entropy over all positions (<PAD> ignored) -> backprop through time -> clip gradient norm to 5 -> Adam step (lr 0.001).
- Per epoch: validation loss. Better -> save `best.pt`. Not better -> halve the LR; 3 epochs without improvement -> stop.
- v5: best val loss **3.615** at epoch 3, stopped at epoch 6; ~106 s/epoch on an RTX 5070 (~11 min total).

9 Generate

generate.py · sampling.py

- Load `best.pt` + `vocab.json` (refuses if the data hash differs). Prime with a 256-token excerpt from the **validation** split.
- Loop 500x: get next-token probabilities -> ban <PAD> <BOS> <EOS> <UNK> <STYLE> -> divide by temperature 0.9 -> random draw -> feed back (hidden state carried, one token per step).
- Writes `<name>_T0.90_seed<seed>_<i>.mid` + `.txt` of tokens. -- seed <seed> reproduces it.

10 Decode and render audio

decode.py · render_audio.py

- Tokens -> notes (each one 16th long) -> `fill_durations` lets each note ring until the next onset (max 4 beats). This is what removes the staccato sound.
- `render_audio.py`: numpy piano synth (decaying partials, reverb, -14 dBFS) -> 44.1 kHz stereo WAV. --instrument piano|epiano|organ.

11 Evaluate

src/evaluate.py

- Scores generated MIDI against real music: in-key 0.884 (real 0.84-0.88, random 0.583), plus variety, rhythm, repetition and copy checks. See page 2.

Parameters, outputs and what the numbers mean

v5 = value in `configs/adl_mixed.yaml` (falls back to `configs/default.yaml`). Change any value without editing a file: `--set section.key=value`. Settings in **data / augment / encoding** are hashed into the cache folder name, so changing them rebuilds the data.

Key	v5	What it does / what the value means
data — what music is read and how		
raw_dir	data/raw/adl	Folder of MIDI; subfolders = styles
include_styles	Classical, Jazz, Blues	Only these subfolders; [] = all
grid	0.25	Quantize step in beats. 0.25 = 16th note, 0.5 = 8th
tempo_map	true	Follow every tempo change, not just the first
min_duration	0.125	Drop notes shorter than this (beats)
min_events	50	Drop pieces with fewer notes
time_signatures	[4/4, 3/4, 2/4]	Keep a piece only if every signature is listed
track_strategy	merge	merge all tracks, or keep the densest
val_split, test_split	0.1, 0.1	Fraction of pieces held out
split_seed	1337	Same seed → same split every run
augment — transposition of the training split		
enabled	true	Turn augmentation on/off
transpose_range	[-6, 6]	Semitone shifts; [-6, 6] = 13 keys per piece
pitch_range	[21, 108]	Piano range; copies leaving it are dropped
encoding — music → tokens (seam 1)		
scheme	note_chord	note_chord interval pitch_duration event
include_duration	false	Put note length in the symbol (true → 363k symbols)
style_tokens	true	Start each piece with <STYLE:x>
run_length_rests	true	One <REST:k> per silence, not k × <REST>
max_rest_run	16	Largest k in one rest token (16 = one 4/4 bar)
min_freq	10	Symbols seen fewer times → <UNK>; 1 = keep all
max_size	0	Cap vocabulary size; 0 = no cap
model — the network (seam 2)		
arch	lstm	lstm transformer
seq_len	256	Context in tokens ≈ 93 beats ≈ 23 bars of 4/4
window_stride	128	Step between windows; 128 = half overlap
window_cover_tail	true	Extra window at each piece's end
embed_dim	256	Size of each symbol's learned vector
hidden_dim	512	LSTM memory width per layer
num_layers	3	Stacked LSTM layers
dropout	0.3	Fraction of activations zeroed in training (fights overfitting)
output	adaptive	full softmax or adaptive (41 % faster, 1/5 memory)
adaptive_cutoffs	[2000, 10000]	Head = 2,000 most common symbols (89 % of targets)
adaptive_div_value	4.0	Each rarer cluster gets 4× narrower vectors
num_heads	8	Transformer only: attention heads

Files it writes

data/processed/<hash>/	*_tokens.npy ids, vocab.json, meta.json (settings, sizes)
runs/<name>/best.pt	Best weights + config + Adam state; last.pt = latest epoch
runs/<name>/history.csv	Per epoch: train_loss, val_loss, val_ppl, lr, seconds, improved (1 = saved)

Key	v5	What it does / what the value means
train — the training loop		
batch_size	48	Windows per step; limited by GPU memory (12 GB)
epochs	60	Upper limit; early stopping ends sooner
lr	0.001	Adam learning rate (step size)
optimizer	adam	adam adamw rmsprop
grad_clip	5.0	Max gradient norm; stops one bad batch wrecking weights
weight_decay	0.0	L2 penalty on weights; 0 = off
patience	3	Stop after this many epochs without improvement
min_delta	0.0	How much val must drop to count as better
lr_schedule	plateau	none plateau (×0.5 when stuck) cosine
lr_factor, lr_patience	0.5, 0	Multiply LR by 0.5 on the first stale epoch
device	auto	GPU if available, else CPU
amp	off	Mixed precision; bf16 hurt quality (5.35 vs 4.80)
seed	1337	Weight init and batch order
resume	false	Continue from the last checkpoint
generate — making new music		
num_tokens	500	Tokens to write per sample (~45 bars)
num_samples	5	Pieces per run (--num-samples)
temperature	0.9	<1 safer and more repetitive, >1 wilder; 0 = always top token
top_k, top_p	0, 0.0	Keep only top k / top p probability mass; 0 = off
seed_source	dataset	Primer from dataset, a midi or an audio file
seed_split	val	Prime from held-out music, so it can't recite training
seed	null	RNG seed; null = random (printed, and in the file name)
duration_fill	next_onset	Notes ring to the next onset; grid = 16th blips
max_fill_beats	4.0	Longest a filled note may ring
program	0	General MIDI instrument in the .mid (0 = piano)

What the result numbers mean

Number	Meaning	v5
val loss	Avg. surprise on held-out music (cross-entropy). Lower = better	3.615
perplexity	e ^{loss} : like choosing among this many symbols	37 / 37,020
in_key_pct	Notes inside the detected key. Random 0.583, real 0.84–0.88	0.884
distinct_n	Share of unique n-note patterns (variety)	0.74
rhythm_div	Variety of gaps between notes (real 0.394)	0.363
repetition_rate	Same note repeated at once (real 0.028)	0.055

Run: `venv\Scripts\python.exe main.py --config configs/adl_mixed.yaml --set name=adl_mixed_lstm_v6 → scripts/render_audio.py data/generated → main.py evaluate --config ...` Quick check: `main.py demo`. Tests: `-m pytest tests -q`. Output lands in `data/generated/`: `.mid` music, `.txt` tokens, `.wav` after rendering.